

XP Diário — Diagrama de Dados e Integração

Estrutura de banco de dados, endpoints da API, relacionamentos e serviços externos

ARQUITETURA GERAL (3 CAMADAS)

FRONTEND (ESTÁTICO)

-  index.html — Login
-  app.html — Dashboard
-  materias.html
-  cronogramas.html
-  tarefas.html
-  metas.html
-  estudos.html
-  relatorio.html
-  turmas.html
-  perfil.html
-  cadastro.html
-  resetar-senha.html

BANCO DE DADOS (NEON POSTGRESQL)

-  usuarios
-  disciplina
-  meta
-  atividade

API BACKEND (NODE.JS + EXPRESS V5)

-  /api/login
-  /api/cadastro
-  /api/materias
-  /api/cronogramas
-  /api/tarefas
-  /api/metast
-  /api/atividades
-  /api/sessoes
-  /api/relatorio
-  /api/conquistas
-  /api/streak
-  /api/progresso
-  /api/turmas
-  /api/notificacoes
-  /api/perfil
-  /api/admin
-  /api/senha

SERVIÇOS EXTERNOS

-  Neon PostgreSQL AWS sa-east-1 (SP), SSL
-  Gmail SMTP Nodemailer, porta 587
-  node-cron Cron a cada 1 min
-  JWT jsonwebtoken, exp. 24h

 conquista

 usuario_conquista

 tarefa

 tarefa_historico

 cronograma_estudo

 sessao_estudo

 turma

 turma_aluno

 notificacoes_config

 bcrypt 10 rounds de hash

 Render Deploy backend (free tier)

TABELAS DO BANCO DE DADOS (13 TABELAS)

usuarios

AUTENTICAÇÃO

PK us_id SERIAL
us_nome VARCHAR(120)
UQ us_email VARCHAR(180)
us_senha_hash VARCHAR(255)
us_tipo ENUM(admin|professor|aluno)
us_pontos_total INT DEFAULT 0
us_tentativas_falhas INT DEFAULT 0
us_bloqueado_ate TIMESTAMP?
us_reset_token_hash TEXT?
us_reset_expira_em TIMESTAMP?
us_criado_em TIMESTAMP DEFAULT now()

disciplina

CONTEÚDO

PK di_id SERIAL
di_disciplina VARCHAR(100)
di_dificuldade VARCHAR(20)?
di_descricao TEXT?
di_cor VARCHAR(20)?
FK di_usuario_id → usuarios

meta

CONTEÚDO

PK me_id SERIAL
me_tipo ENUM(diaria|semanal|mensal)
me_tempo_min INT CHECK(> 0)
FK me_disciplina → disciplina?
me_data_inicio DATE?
FK me_usuario_id → usuarios

atividade

CONTEÚDO

PK at_id SERIAL
FK at_disciplina → disciplina
at_tempo_min INT CHECK(> 0)
at_tarefas_concluidas INT DEFAULT 0
at_data DATE DEFAULT today
at_descricao VARCHAR(300)?
FK at_usuario_id → usuarios
at_criado_em TIMESTAMP DEFAULT now()

conquista

GAMIFICAÇÃO

PK co_id SERIAL

usuario_conquista

GAMIFICAÇÃO

PK uc_id SERIAL

UQ co_codigo VARCHAR(50)
co_titulo VARCHAR(120)
co_descricao VARCHAR(255)
co_criterio_tipo VARCHAR(40)
co_criterio_valor INT CHECK(> 0)
co_criado_em TIMESTAMP DEFAULT now()

FK uc_usuario_id → usuarios
FK uc_conquista_id → conquista
UQ (usuario_id, conquista_id)
uc_desbloqueado_em TIMESTAMP

tarefa

TAREFAS

PK ta_id SERIAL
ta_titulo VARCHAR(160)
ta_descricao TEXT?
ta_prazo DATE?
FK ta_usuario_id → usuarios
FK ta_disciplina_id → disciplina?
ta_status ENUM(pendente|concluida)
ta_concluida_em TIMESTAMP?
ta_criado_em / ta_atualizado_em

tarefa_historico

TAREFAS

PK th_id SERIAL
FK th_tarefa_id → tarefa
FK th_usuario_id → usuarios
th_acao VARCHAR(40)
th_titulo_snapshot VARCHAR
th_status_anterior / th_status_novo
th_criado_em TIMESTAMP DEFAULT now()

cronograma_estudo

AGENDA

PK cr_id SERIAL
FK cr_usuario_id → usuarios
FK cr_disciplina → disciplina
cr_data DATE
cr_horario_inicio TIME
cr_duracao_min INT CHECK(> 0)
UQ (usuario, disciplina, data, hora)

sessao_estudo

AGENDA

PK se_id SERIAL
FK se_usuario_id → usuarios
FK se_disciplina → disciplina?
se_status ENUM(iniciada|pausada|encerrada)
se_segundos_focados INT DEFAULT 0
se_ultimo_inicio TIMESTAMP
se_inicio / se_fim TIMESTAMP

turma

SOCIAL

PK tu_id SERIAL
tu_nome VARCHAR(100)
UQ tu_codigo VARCHAR(8)
FK tu_professor_id → usuarios
tu_ativa BOOLEAN DEFAULT true
tu_criada_em TIMESTAMPTZ DEFAULT now()

turma_aluno

SOCIAL

PK ta_id SERIAL
FK ta_turma_id → turma
FK ta_aluno_id → usuarios
UQ (turma_id, aluno_id)
ta_entrou_em TIMESTAMPTZ DEFAULT now()

notificacoes_config

CONFIG

PK nc_id SERIAL
FK **UQ** nc_usuario_id → usuarios
nc_ativo BOOLEAN DEFAULT false
nc_horario TIME DEFAULT '08:00'
nc_fuso_horario VARCHAR(60)
nc_ultimo_envio DATE?
nc_atualizado_em TIMESTAMP DEFAULT now()

RELACIONAMENTOS

usuarios 1 : N **disciplina**

Um usuário cria e gerencia várias disciplinas

usuarios 1 : N **meta**

Um usuário define várias metas de estudo (diária/semanal/mensal)

usuarios 1 : N **atividade**

Cada sessão encerrada gera um registro em atividade

disciplina 1 : N **atividade**

Cada atividade está associada a uma disciplina

usuarios N : M **conquista**

Via tabela usuario_conquista — um usuário pode desbloquear várias conquistas

usuarios 1 : N **tarefa**

Um usuário gerencia várias tarefas em diferentes status

tarefa 1 : N **tarefa_historico**

Cada criação/edição/conclusão/exclusão gera um log imutável

usuarios 1 : N **cronograma_estudo**

Um usuário agenda várias sessões no calendário

usuarios 1 : N **sessao_estudo**

Um usuário pode ter várias sessões ao vivo (uma ativa por vez)

professor (usuarios) 1 : N **turma**

Professor cria e gerencia turmas com código único de ingresso

turma N : M **aluno (usuarios)**

Via turma_aluno — alunos entram em turmas usando o código único

usuarios 1 : 1 **notificacoes_config**

Uma configuração de notificação por usuário (upsert na criação)

PRINCIPAIS ENDPOINTS DA API

MÉTODO	ROTA	DESCRIÇÃO	AUTH
POST	/api/login	Autentica usuário, retorna JWT	—
POST	/api/cadastro	Cria novo usuário (aluno/professor)	—
POST	/api/senha/recuperar	Envia e-mail de reset de senha	—
POST	/api/senha/redefinir	Redefine senha com token válido	—
GET	/api/materias	Lista disciplinas do usuário logado	JWT
POST	/api/materias	Cria nova disciplina	JWT
PUT	/api/materias/:id	Atualiza disciplina existente	JWT
DEL	/api/materias/:id	Exclui disciplina (cascade)	JWT
GET	/api/tarefas	Lista tarefas com resumo e histórico	JWT

POST	/api/tarefas	Cria tarefa + log histórico	JWT
PATCH	/api/tarefas/:id/concluir	Marca tarefa como concluída	JWT
POST	/api/sesoes/iniciar	Inicia sessão ao vivo	JWT
POST	/api/sesoes/pausar	Pausa sessão ativa	JWT
POST	/api/sesoes/retomar	Retoma sessão pausada	JWT
POST	/api/sesoes/encerrar	Encerra sessão, calcula XP, grava atividade	JWT
GET	/api/conquistas	Lista conquistas (bloqueadas + desbloqueadas)	JWT
GET	/api/streak	Retorna streak atual e histórico de dias	JWT
GET	/api/progresso	Progresso por disciplina (XP, metas, atividades)	JWT
GET	/api/relatorio	Relatório consolidado com filtro de período	JWT
POST	/api/turmas	Cria turma (professor only)	JWT
POST	/api/turmas/entrar	Aluno entra em turma via código	JWT
GET	/api/turmas/:id/alunos	Lista alunos de uma turma (professor only)	JWT
GET	/api/notificacoes/config	Configurações de notificação do usuário	JWT
PUT	/api/notificacoes/config	Atualiza horário e ativa/desativa notificações	JWT
GET	/api/admin/usuarios	Lista todos os usuários (admin only)	JWT + admin

SERVIÇOS E INTEGRAÇÕES EXTERNAS



Neon PostgreSQL

Banco de dados serverless hospedado na AWS **sa-east-1** (São Paulo). Conexão SSL obrigatória via driver pg (node-postgres). Auto-migrate ao iniciar o servidor.

aws sa-east-1 SSL / TLS Serverless



Gmail SMTP (Nodemailer)

Envio de e-mails de recuperação de senha e lembretes diários de estudo. Host: smtp.gmail.com, Porta: 587 (STARTTLS). Credenciais via App Password.

STARTTLS App Password Nodemailer



node-cron

Executa a cada minuto para verificar notificações. Aplica fuso horário individual por usuário. Envia e-mail no máximo 1x/dia. Keep-alive via ping interno a cada 10 min.



JWT (jsonwebtoken)

Autenticação stateless. Token contém id, email, nome e tipo do usuário. Expiração em **24h**. Verificado em middleware para todas as rotas protegidas.

HS256 24h TTL Stateless

Cron * * * * *

Timezone-aware



bcrypt

Hash de senhas com **10 rounds** de salt. Proteção adicional: bloqueio de conta após 5 tentativas falhas (30 minutos). Tokens de reset com validade de 1h.

10 salt rounds

Brute-force lock



Render (Deploy)

Hospedagem do backend Node.js (free tier). Servidor hiberna após inatividade. Keep-alive implementado via auto-ping a cada 10 minutos para manter o serviço acordado.

Free Tier

Keep-alive

Auto-deploy